

Hands-on tutorial — Local RAG + Local LLM (no cloud, no API keys)

Everything here runs **on your own laptop**. No accounts, no API keys, no data leaves your machine. Our knowledge base is the official **ISTQB CTFL syllabus PDF** in [docs/](#). We build up in three small, readable Python files.

The plan for the day

Step	What you do	File
0	See how text becomes tokens (in the browser)	<i>(website, no code)</i>
1	Run a local LLM (Llama 3B) and ask it an ISTQB question — see it answer from memory (and get it wrong)	1_local_llm.py
2	Build RAG retrieval : pull the syllabus PDF and find the exact part that answers the question — no AI model	2_rag.py
3	Combine them: retrieve from the syllabus, then ask the model → a correct, grounded answer	3_rag_plus_llm.py

The three steps deliberately ask the **same question** ("what are the seven testing principles?"). In Step 1 the model answers from memory and gets it wrong; in Step 3, fed the real syllabus, it answers correctly. That contrast is the whole point.

Setup (once, before we start)

Use the **latest version of Python** (<https://www.python.org>). Every command below is something you can copy-paste; the *check* commands confirm each piece is working before you move on.

On macOS/Linux you may need [python3](#) and [pip3](#) instead of [python](#) / [pip](#). On Windows use [python](#) and [pip](#).

1. Check Python is installed

```
python --version
```

You should see a version number (e.g. [Python 3.x.x](#)). If you get an error, install Python from <https://www.python.org> and reopen your terminal.

2. Check pip is installed

```
pip --version
```

(**pip -V** does the same thing.) pip is Python's package installer — it comes with Python. You should see a version and a path.

3. Activate the shared virtual environment (created once, at the repo root)

A "virtual environment" is a private folder of packages just for this project, so we don't touch your system Python. We use **one environment for the whole workshop**, created at the **repo root** (one level up from here).

If you already set it up in the [GenAI_Intro/ lab](#), just activate it — you don't need a new one. Otherwise create it now:

```
cd ..          # go to the repo root if you aren't already there
python -m venv venv  # skip this line if the venv folder already exists
```

Then activate it:

```
source venv/bin/activate  # macOS / Linux
venv\Scripts\activate     # Windows
```

After activating, your prompt shows (**venv**). Confirm it's the right Python:

```
python --version
```

To leave a venv later, type **deactivate** (same on macOS / Linux / Windows).

4. Install the project packages (one file at the repo root)

```
pip install -r requirements.txt
```

Check they installed:

```
pip list
```

You should see **sentence-transformers**, **numpy**, **pypdf** and **ollama** in the list.

Then come back into this folder to run the steps below:

```
cd tutorial
```

5. Install Ollama (needed for Steps 1 and 3)

1. Download and install Ollama (free): <https://ollama.com/download> (macOS / Windows: run the installer; it then runs in the background.)
2. Check Ollama is installed:

```
ollama --version
```

3. Download the model (~2 GB, one time):

```
ollama pull llama3.2:3b
```

4. Check the model works (no Python needed):

```
ollama list                # the model
should be listed
ollama run llama3.2:3b "Say hello in one sentence." # it should
answer
```

If the last command prints a sentence, your local model is running. Type `/bye` to leave the `ollama run` chat.

Step 0 — See tokenization in your browser (no code)

Open the OpenAI tokenizer and type a sentence: <https://platform.openai.com/tokenizer> (or <https://tiktokenizer.vercel.app>)

Watch your text get split into **tokens** (small pieces). This is the first thing every model does with text. In Step 2 you'll see those pieces turned into **numbers** (vectors) for real.

Step 1 — Run and call a local LLM

```
python 1_local_llm.py
```

Asks the local Llama 3B model: *"what are the seven ISTQB testing principles?"* The model answers from its **general memory** — it has not seen the syllabus, so it confidently makes up the wrong principles, and we can't trace its answer to a source. That's the problem RAG solves.

Step 2 — RAG retrieval, no AI model

```
python 2_rag.py
```

This reads the syllabus PDF in `docs/`, cuts it into chunks (one per document **section**, so a list of items is never split in half), turns each chunk into a vector with a small local model, and **saves all the vectors to `embeddings.json`**. Then it searches for the **same topic** using **cosine similarity** — with **no LLM and no API**.

Tip you'll see in the code: we *search* with a few focused key words ("seven testing principles"), even though we *ask the model* the full sentence. Retrieval works best with key terms.

The first run is slower (it embeds the whole PDF once); later runs reuse `embeddings.json` and are fast. To force a rebuild, delete `embeddings.json`.

`embeddings.json` is our simple, readable "vector database". In a real project you'd store these vectors in a proper vector database — e.g. **PostgreSQL + pgvector**, **MongoDB Atlas Vector Search**, or engines like Chroma / Qdrant / Pinecone / FAISS — which search huge numbers of vectors quickly.

Step 3 — RAG + local LLM together

```
python 3_rag_plus_llm.py
```

Retrieves the relevant chunks from the syllabus (Step 2), pastes them into the prompt as context, and asks the local model (Step 1) the **same question** — now answered **using only the syllabus**. Compare this answer to Step 1: this is why Retrieval-Augmented Generation matters.

Using your own documents

Drop any `.pdf` (or `.md`) file into `docs/`, delete `embeddings.json` so it rebuilds, and re-run — the code picks up every PDF/Markdown file automatically.